FastAPI Reference

Templating - `Jinja2Templates`

You can use the `Jinja2Templates` class to render Jinja templates.

Read more about it in the FastAPI docs for Templates.

You can import it directly from `fastapi.templating`:

```
from fastapi.templating import Jinja2Templates
```

class fastapi.templating.Jinja 2 Templates

```
Jinja2Templates(
    directory: (
        str | PathLike[str] | Sequence[str | PathLike[str]]
    ),
    *,
    context_processors: (
        list[Callable[[Request], dict[str, Any]]] | None
    ) = None,
    **env_options: Any
)
```

```
Jinja2Templates(
    *,
    env: Environment,
    context_processors: (
        list[Callable[[Request], dict[str, Any]]] | None
    ) = None
)
```

```
Jinja2Templates(
    directory=None,
    *,
    context_processors=None,
    env=None,
    **env_options
)
```

templates = Jinja 2 Templates("templates")

return templates.TemplateResponse("index.html", {"request": request})

Ask AI

Source code in `starlette/templating.py`

```
def __init__(
    self,
    directory: str | PathLike[str] | Sequence[str | PathLike[str]] | None = None,
    *,
    context_processors: list[Callable[[Request], dict[str, Any]]] | None = None,
    env: jinja2.Environment | None = None,
    **env_options: Any,
) -> None:
    if env_options:
        warnings.warn(
            "Extra environment options are deprecated. Use a preconfigured jinja2.Environment
instead.",
            DeprecationWarning,
        )
    assert jinja2 is not None, "jinja2 must be installed to use Jinja2Templates"
    assert bool(directory) ^ bool(env), "either 'directory' or 'env' arguments must be passed"
    self.context_processors = context_processors or []
    if directory is not None:
        self.env = self._create_env(directory, **env_options)
    elif env is not None:  # pragma: no branch
        self.env = env

    self._setup_env_defaults(self.env)
```

attr context_processors `instance-attribute`

```
context_processors = context_processors or []
```

attr env `instance-attribute`

```
env = _create_env(directory, **env_options)
```

meth get_template

```
get_template(name)
```

Source code in `starlette/templating.py`

```
def get_template(self, name: str) -> jinja2.Template:
    return self.env.get_template(name)
```

meth TemplateResponse

```
TemplateResponse(
    request: Request,
    name: str,
    context: dict[str, Any] | None = None,
    status_code: int = 200,
    headers: Mapping[str, str] | None = None,
    media_type: str | None = None,
    background: BackgroundTask | None = None,
) -> _TemplateResponse
```

```
TemplateResponse(
    name: str,
    context: dict[str, Any] | None = None,
```

```
    status_code: int = 200,
    headers: Mapping[str, str] | None = None,
    media_type: str | None = None,
    background: BackgroundTask | None = None,
) -> _TemplateResponse
```

```
TemplateResponse(*args, **kwargs)
```

Source code in `starlette/templating.py`

```
def TemplateResponse(self, *args: Any, **kwargs: Any) -> _TemplateResponse:
    if args:
        if isinstance(args[0], str):  # the first argument is template name (old style)
            warnings.warn(
                "The `name` is not the first parameter anymore. "
                "The first parameter should be the `Request` instance.\n"
                'Replace `TemplateResponse(name, {"request": request})` by
`TemplateResponse(request, name)`.',
                DeprecationWarning,
            )

            name = args[0]
            context = args[1] if len(args) > 1 else kwargs.get("context", {})
            status_code = args[2] if len(args) > 2 else kwargs.get("status_code", 200)
            headers = args[3] if len(args) > 3 else kwargs.get("headers")
            media_type = args[4] if len(args) > 4 else kwargs.get("media_type")
            background = args[5] if len(args) > 5 else kwargs.get("background")

            if "request" not in context:
                raise ValueError('context must include a "request" key')
            request = context["request"]
        else:  # the first argument is a request instance (new style)
            request = args[0]
            name = args[1] if len(args) > 1 else kwargs["name"]
            context = args[2] if len(args) > 2 else kwargs.get("context", {})
            status_code = args[3] if len(args) > 3 else kwargs.get("status_code", 200)
            headers = args[4] if len(args) > 4 else kwargs.get("headers")
            media_type = args[5] if len(args) > 5 else kwargs.get("media_type")
            background = args[6] if len(args) > 6 else kwargs.get("background")
    else:  # all arguments are kwargs
        if "request" not in kwargs:
            warnings.warn(
                "The `TemplateResponse` now requires the `request` argument.\n"
                'Replace `TemplateResponse(name, {"context": context})` by
`TemplateResponse(request, name)`.',
                DeprecationWarning,
            )
            if "request" not in kwargs.get("context", {}):
                raise ValueError('context must include a "request" key')

        context = kwargs.get("context", {})
        request = kwargs.get("request", context.get("request"))
        name = cast(str, kwargs["name"])
        status_code = kwargs.get("status_code", 200)
        headers = kwargs.get("headers")
        media_type = kwargs.get("media_type")
        background = kwargs.get("background")

    context.setdefault("request", request)
    for context_processor in self.context_processors:
        context.update(context_processor(request))

    template = self.get_template(name)
    return _TemplateResponse(
        template,
        context,
        status_code=status_code,
        headers=headers,
        media_type=media_type,
        background=background,
    )
```